

Reviewer Pack — Symmetric Group Fourier Coefficients Inverse Proportional to...

Entry 567baddf9ba5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 01:48:40 UTC

Symmetric Group Fourier Coefficients Inverse Proportional to ACC⁰ Circuit Size for Sipser Function

Entry ID: 567baddf9ba5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 01:48:40 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Finite Groups **Field B** (complexity object): ACC⁰ Circuit Size for Sipser Function

Statement:

For the Sipser function S_n on n variables, the maximum absolute Fourier coefficient over the symmetric group S_n 's irreducible representations is $\Theta(1/k)$, where k is the minimal ACC⁰ circuit size computing S_n . This holds for all $n \geq 2$.

Rationale (proposer's reasoning):

Symmetric group representations capture combinatorial symmetries of permutation-based functions like Sipser's. Fourier coefficients over these representations quantify 'non-abelian harmonic analysis' of the function, potentially revealing structural obstructions to efficient ACC⁰ computation.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4b5c17751a6c3da2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def sipser_function(n, x):
        return sum(x[i] for i in range(n)) % 2

    def young_tableau_characters(n):
        # Generate all partitions of n and their Young tableaux characters
        partitions = []
        def partition(n, k, acc=[]):
            if len(acc) == k:
                partitions.append(acc)
                return
            for i in range(n, -1, -1):
                partition(i, k-1, acc + [i])
        partition(n, n)

        characters = {}
        for part in partitions:
            char = 1
            for i in range(1, len(part)):
                char *= math.factorial(sum(part[:i])) // (math.prod(math.factorial(x) for x in part[:i]))
            characters[tuple(sorted(part))] = char

        return characters

    def fourier_coefficient(n, k):
        # Compute the Fourier coefficient for a given partition
        characters = young_tableau_characters(n)
        coeff = 0
        for perm in set(itertools.permutations(range(n))):
            sign = (-1) ** sum(perm[i] > perm[j] for i, j in combinations(range(n), 2))
            char = characters[tuple(sorted([perm.index(i) for i in range(k)])
            coeff += sign * char
        return abs(coeff / math.factorial(n))
```

```

def acc0_circuit_size(n):
    # Placeholder function to return a known ACC0 circuit size
    # This is a dummy implementation and should be replaced with actual computation
    return n

max_coeff = 0
for k in range(1, 2**n + 1):
    coeff = fourier_coefficient(n, k)
    if coeff > max_coeff:
        max_coeff = coeff

circuit_size = acc0_circuit_size(n)

return {
    "metric_name": "r_squared",
    "metric_value": max_coeff ** 2 / (circuit_size ** 2),
    "instances_tested": n,
    "conjecture_holds": False if max_coeff == 0 else True,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [677, 727, 773, 821, 877, 929] # Default list of pr

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if not results:
        raise ValueError("No trials were executed. Ensure run_trial is implemented correctly.")

    mean_r_squared = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_r_squared} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE no seeds supported the conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_136fa469.py", line 80, in <module>

```
result = run_trial(seed)
^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_136fa469.py", line 59, in run_trial
    for k in range(1, 2**n + 1):
        ^
```

NameError: name 'n' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'n' before producing results | next: Fix the NameError in test code by defining 'n' before usage

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111033
2	propose	ollama_remote	qwen3:8b	0	0	84354
3	propose	ollama_remote	qwen3:8b	0	0	58197
4	preregistration	ollama_remote	qwen3:8b	0	0	25052
5	novelty	ollama_remote	qwen3:8b	0	0	20561
6	novelty	ollama_remote	qwen3:8b	0	0	14110
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16547
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9288
9	judge	ollama_remote	qwen3:8b	0	0	15889

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 355032 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/567baddf9ba5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/567baddf9ba5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/567baddf9ba5.tar.gz` (if generated)